1. Direct Sorting

Introduction

The objective was to compare and analyse three direct sorting methods, namely bubble sort, insertion sort (more specifically, insertion sort with binary-search insertion) and selection sort.

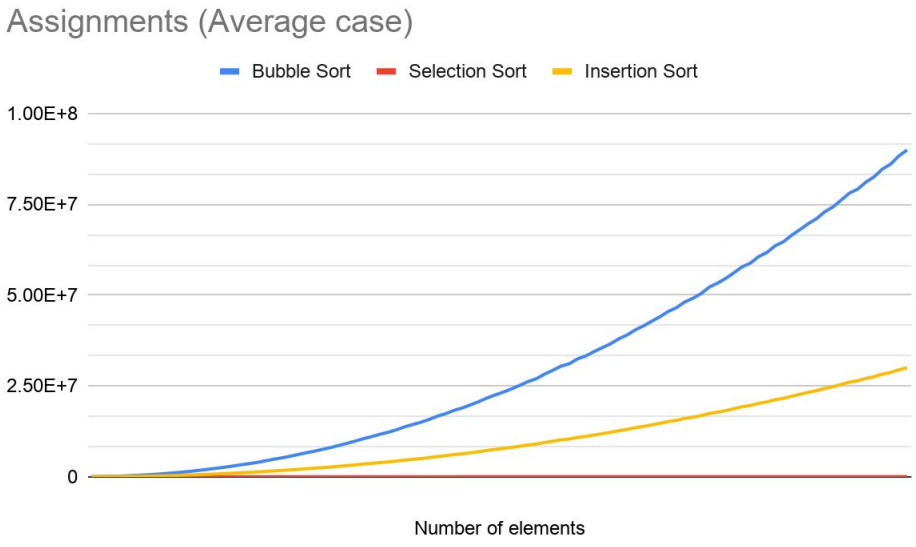
To this end, these algorithms have been implemented in C such that they count the number of comparisons and assignments required to sort an array in average, best, and worst cases. These values were then collected into a spreadsheet so that the algorithms can be compared.

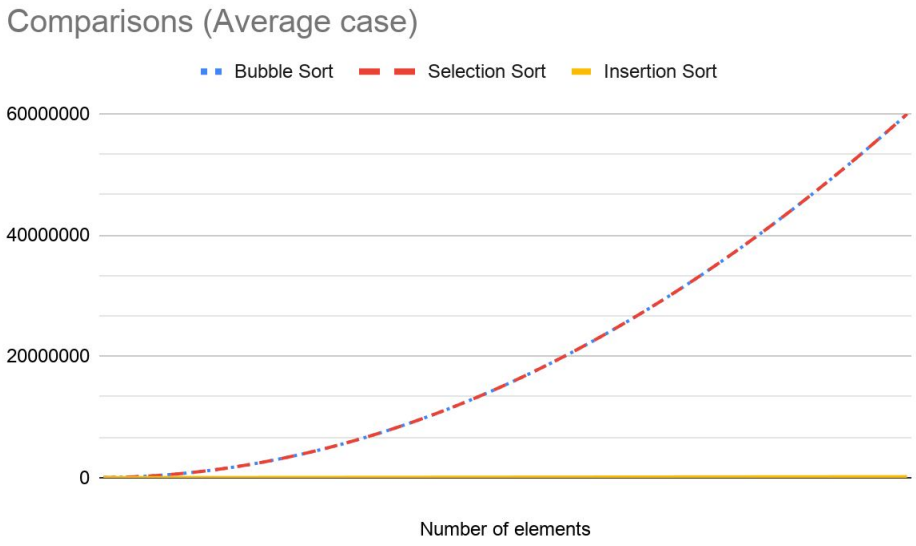
To prove correctness, the main program will also print a demonstration on a set of small arrays.

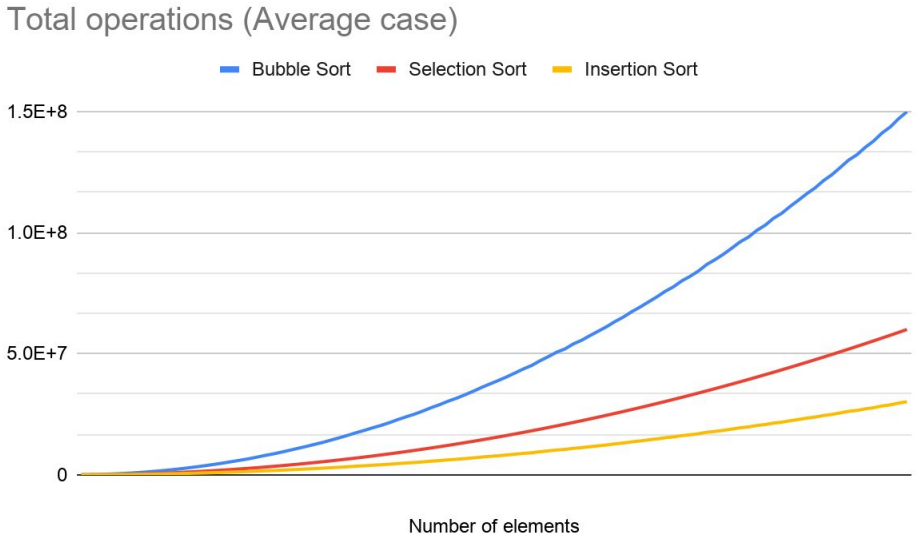
Results

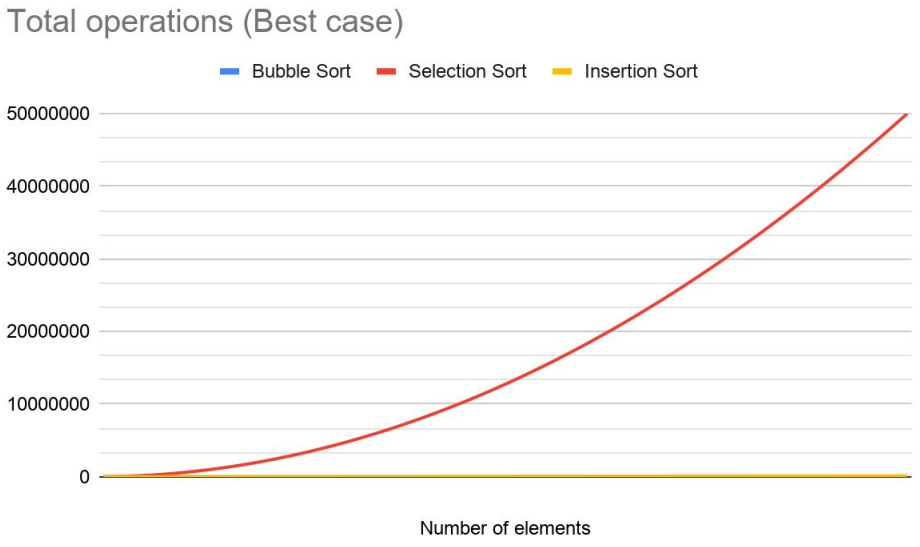
The following is an excerpt from the results. The complete results are attached as a CSV file, while a wider range of charts is presented in the annex at the end of this document.

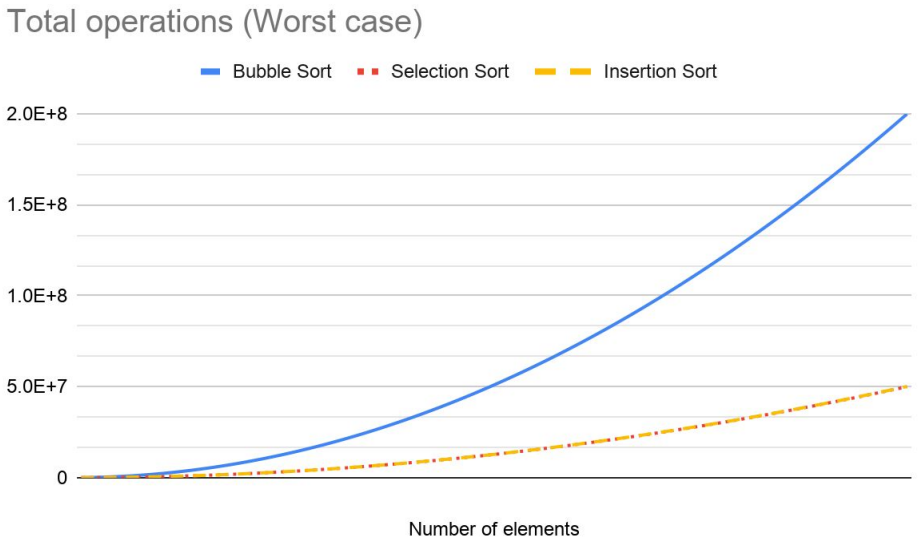
Number of Elements	Bubble Sort Operations	Selection Sort Operations	Insertion Sort Operations
1000	**1.49E+6** (8.92E+5 Assign and 5.98E+5 Compare)	**6.03E+5** (3.57E+3 Assign and 5.99E+5 Compare)	**3.12E+5** (3.00E+5 Assign and 1.20E+4 Compare)
2000	**5.98E+6** (3.58E+6 Assign and 2.40E+6 Compare)	**2.41E+6** (7.17E+3 Assign and 2.40E+6 Compare)	**1.22E+6** (1.20E+6 Assign and 2.63E+4 Compare)
3000	**1.34E+7** (8.02E+6 Assign and 5.40E+6 Compare)	**5.41E+6** (1.08E+4 Assign and 5.40E+6 Compare)	**2.72E+6** (2.68E+6 Assign and 4.16E+4 Compare)
4000	**2.40E+7** (1.44E+7 Assign and 9.59E+6 Compare)	**9.61E+6** (1.44E+4 Assign and 9.60E+6 Compare)	**4.87E+6** (4.81E+6 Assign and 5.75E+4 Compare)
5000	**3.75E+7** (2.25E+7 Assign and 1.50E+7 Compare)	**1.50E+7** (1.80E+4 Assign and 1.50E+7 Compare)	**7.60E+6** (7.53E+6 Assign and 7.37E+4 Compare)
6000	**5.41E+7** (3.25E+7 Assign and 2.16E+7 Compare)	**2.16E+7** (2.16E+4 Assign and 2.16E+7 Compare)	**1.09E+7** (1.08E+7 Assign and 9.03E+4 Compare)
7000	**7.35E+7** (4.41E+7 Assign and 2.94E+7 Compare)	**2.94E+7** (2.52E+4 Assign and 2.94E+7 Compare)	**1.48E+7** (1.47E+7 Assign and 1.07E+5 Compare)
8000	**9.62E+7** (5.78E+7 Assign and 3.84E+7 Compare)	**3.84E+7** (2.88E+4 Assign and 3.84E+7 Compare)	**1.94E+7** (1.93E+7 Assign and 1.24E+5 Compare)
9000	**1.22E+8** (7.30E+7 Assign and 4.86E+7 Compare)	**4.86E+7** (3.24E+4 Assign and 4.86E+7 Compare)	**2.45E+7** (2.43E+7 Assign and 1.41E+5 Compare)
10000	**1.50E+8** (9.00E+7 Assign and 6.00E+7 Compare)	**6.00E+7** (3.60E+4 Assign and 6.00E+7 Compare)	**3.02E+7** (3.00E+7 Assign and 1.59E+5 Compare)


Assignments (Average case)
Bubble Sort
Selection Sort
Insertion Sort
1.00E+8
7.50E+7
5.00E+7
2.50E+7
0
Number of elements


Comparisons (Average case)
Bubble Sort
Selection Sort
Insertion Sort
60000000
40000000
20000000
0
Number of elements


Total operations (Average case)
Bubble Sort
Selection Sort
Insertion Sort
1.5E+8
1.0E+8
5.0E+7
0
Number of elements





Discussion

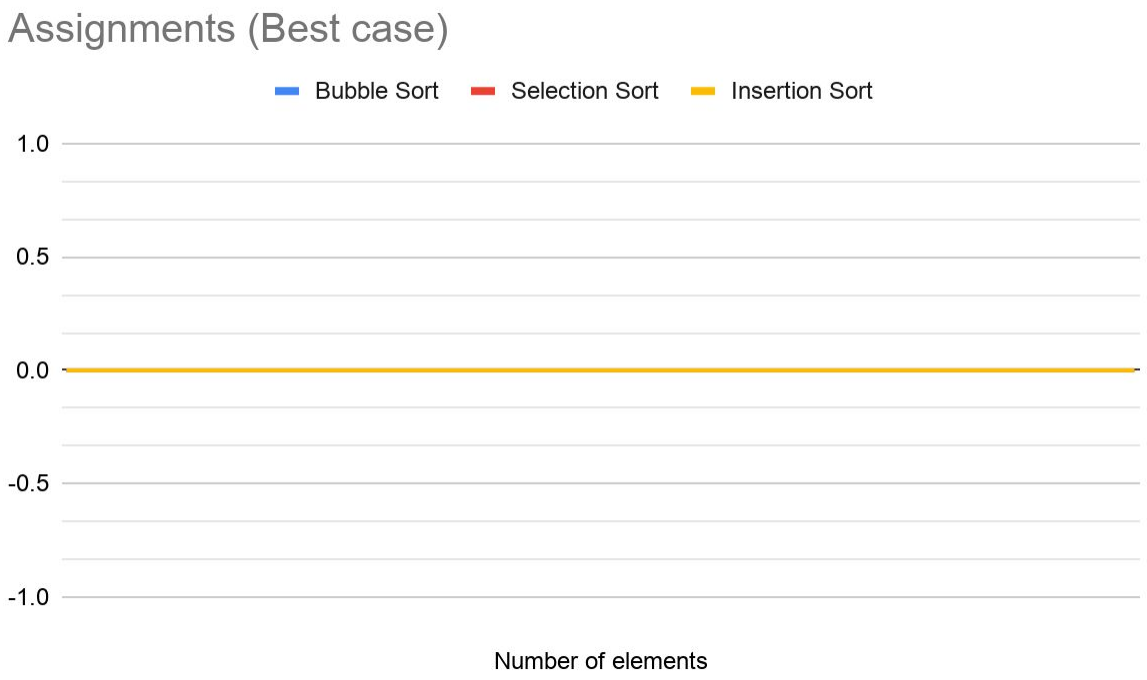
As can be seen from the results, insertion sort tends to perform the best on average in terms of total operations, while bubble sort tends to perform by far the worst.
While bubble sort excels in its best-case scenario, that is simply because its implementation inherently “checks” if the array is already sorted; if this case is likely, a such check can be trivially added to the other algorithms as well.
In the best-case scenario, selection sort is the weakest, as it must still perform all the comparisons to ensure the array is sorted. Selection sort, however, has the benefit of not performing much poorer in the worst case than in the average case for the same reason, so that in a near-worst-case scenario, selection and insertion sort perform similarly.

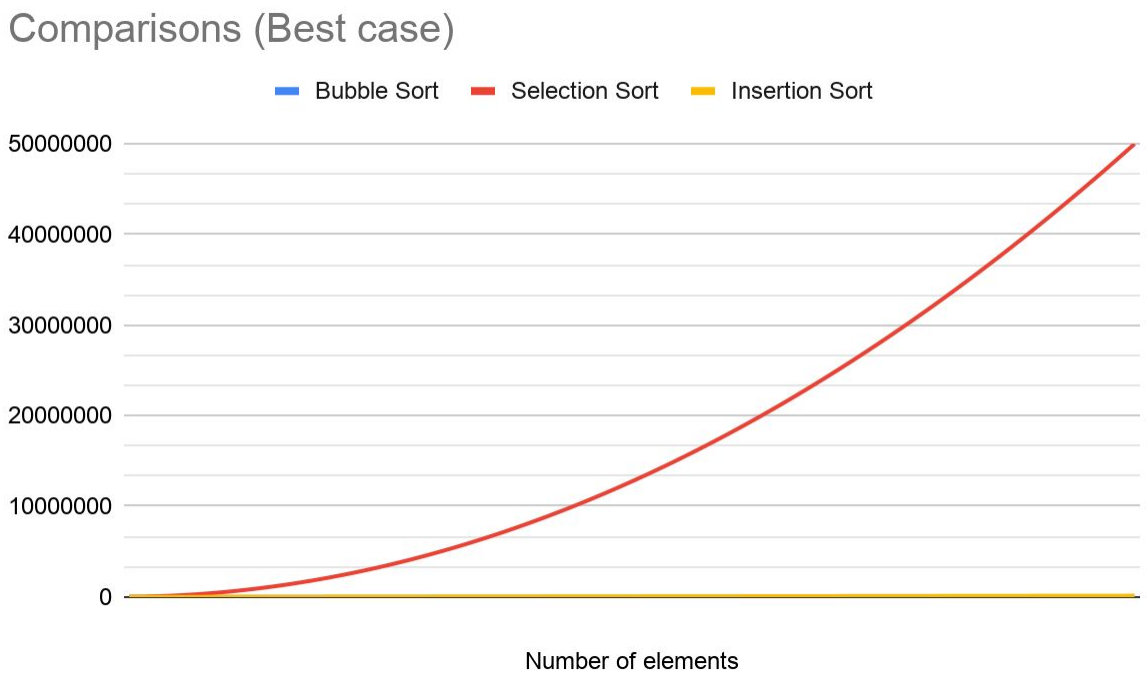
Note that for the above data, swapping two elements has been done using an auxiliary variable rather than using the XOR method, so each swap has been counted as three assignments.

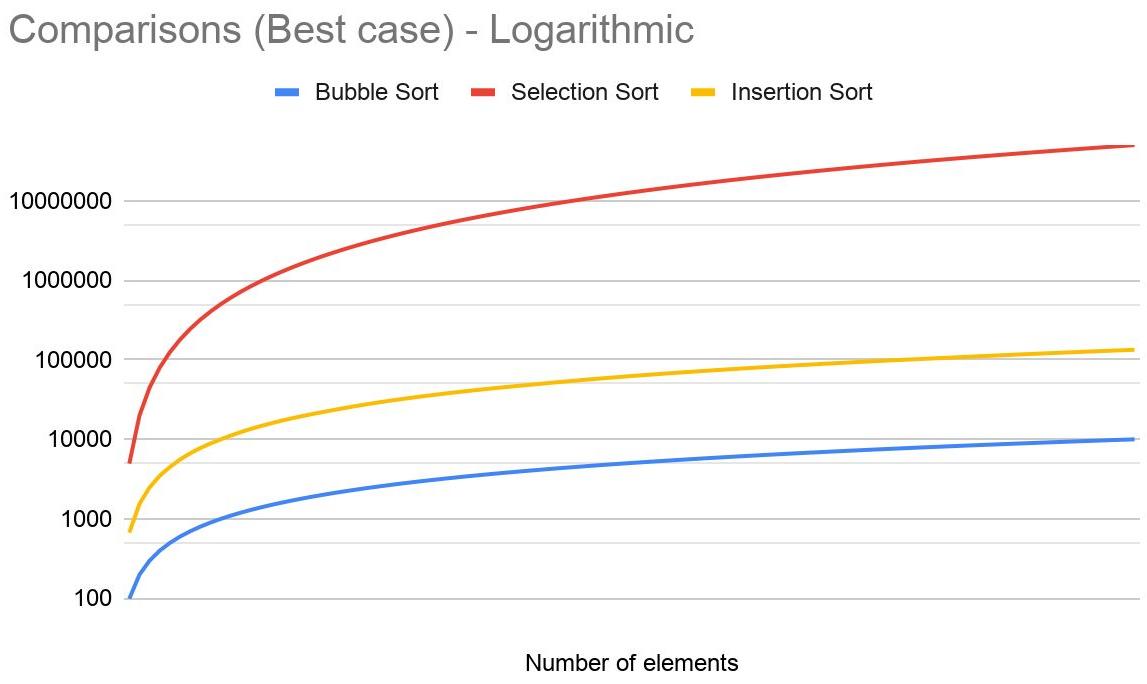
If this were changed so that each swap counts as only two assignments, there would be a slight improvement in the performance of bubble sort (though still not enough to make it competitive).

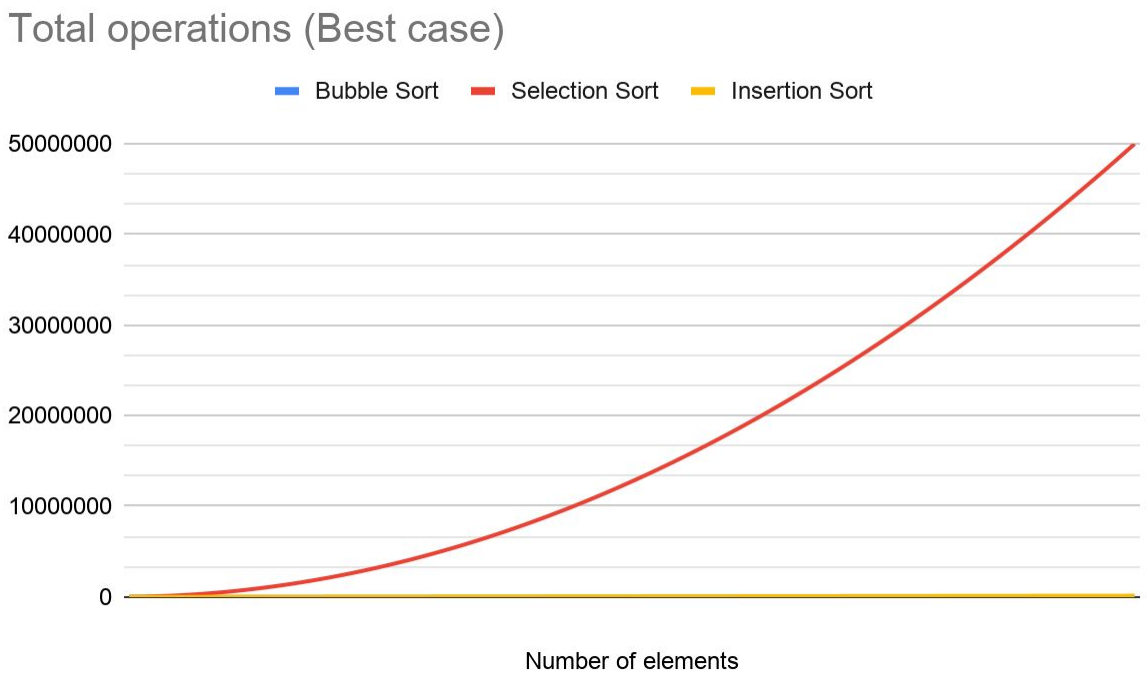
Finally, in terms of stability, both bubble sort and insertion sort are stable. In the case of bubble sort it is because an element will never “bubble up” past an equal other element. In the case of insertion sort, it is because elements are inserted in ascending order and an element will always be inserted at the rightmost position that is valid (that is, elements encountered later will always be placed after equal elements encountered earlier). Selection sort, by comparison, is not stable because a large element at the start of the array might be swapped with a small one at the end of it, such as in the case of 7 5 7 1, where the order of the 7 elements will be reversed.

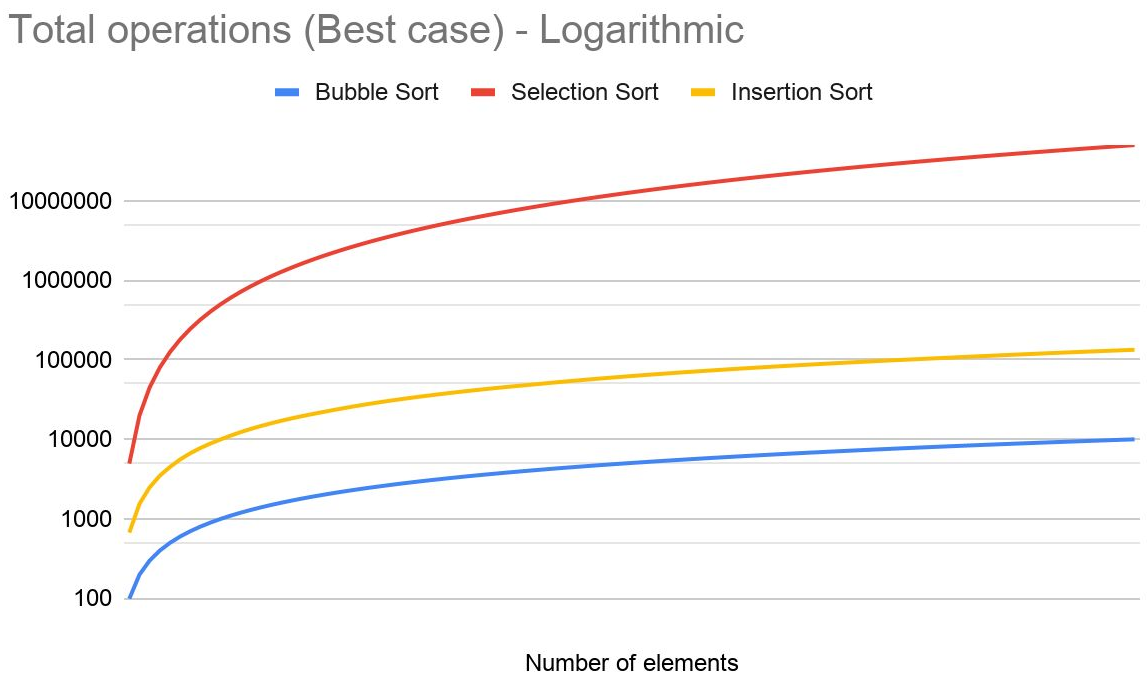
Annex: Additional Charts

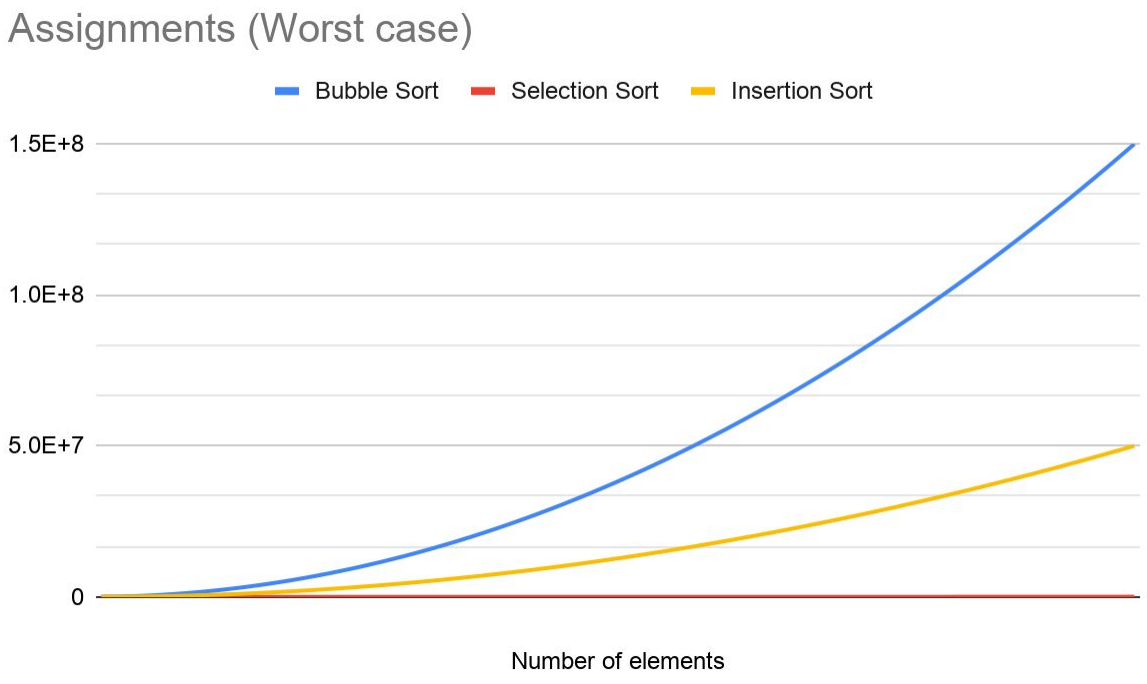


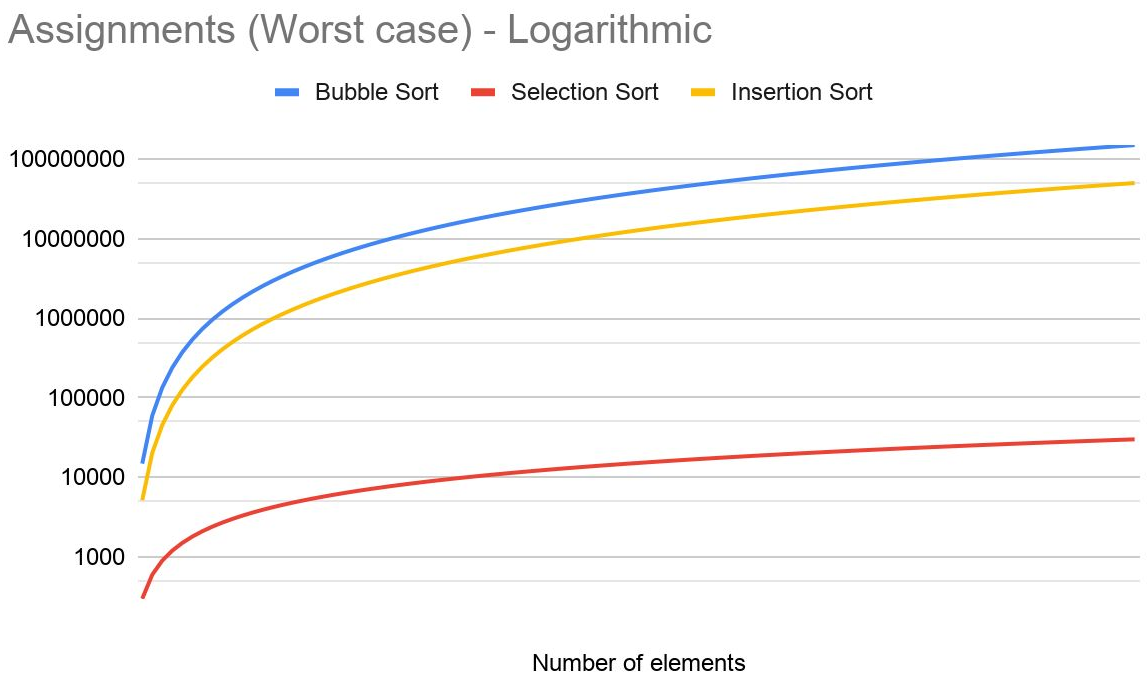

Comparisons (Best case)
Bubble Sort
Selection Sort
Insertion Sort
50000000
40000000
30000000
20000000
10000000
0
Number of elements

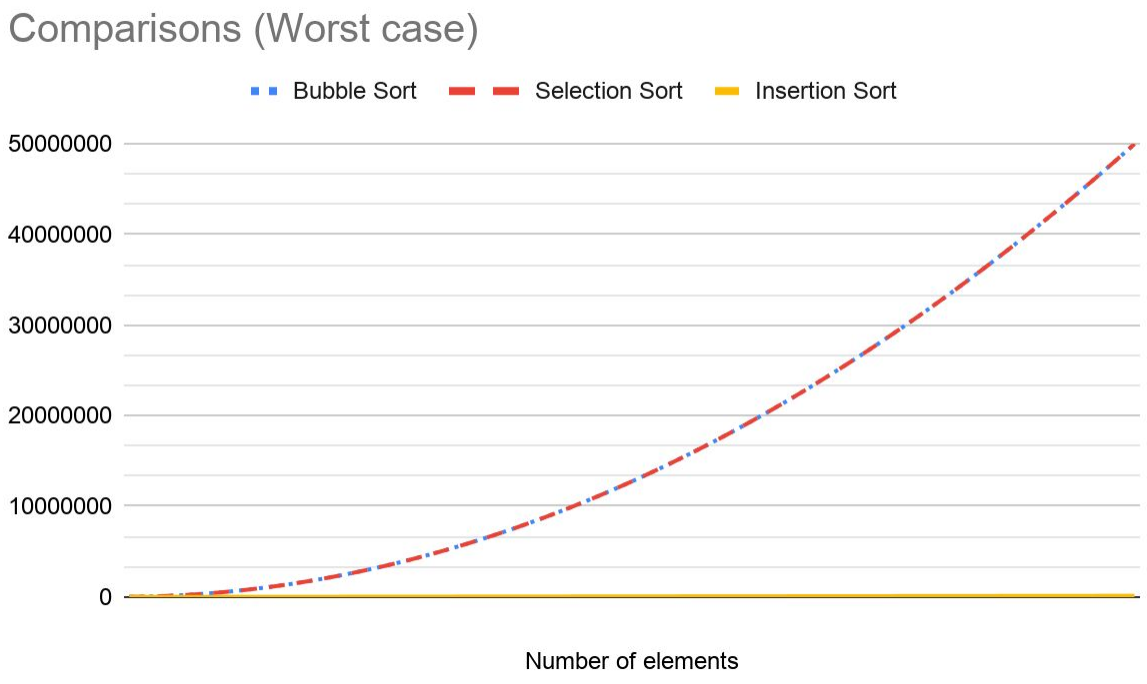

Comparisons (Best case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
10000000
1000000
100000
10000
1000
100
Number of elements

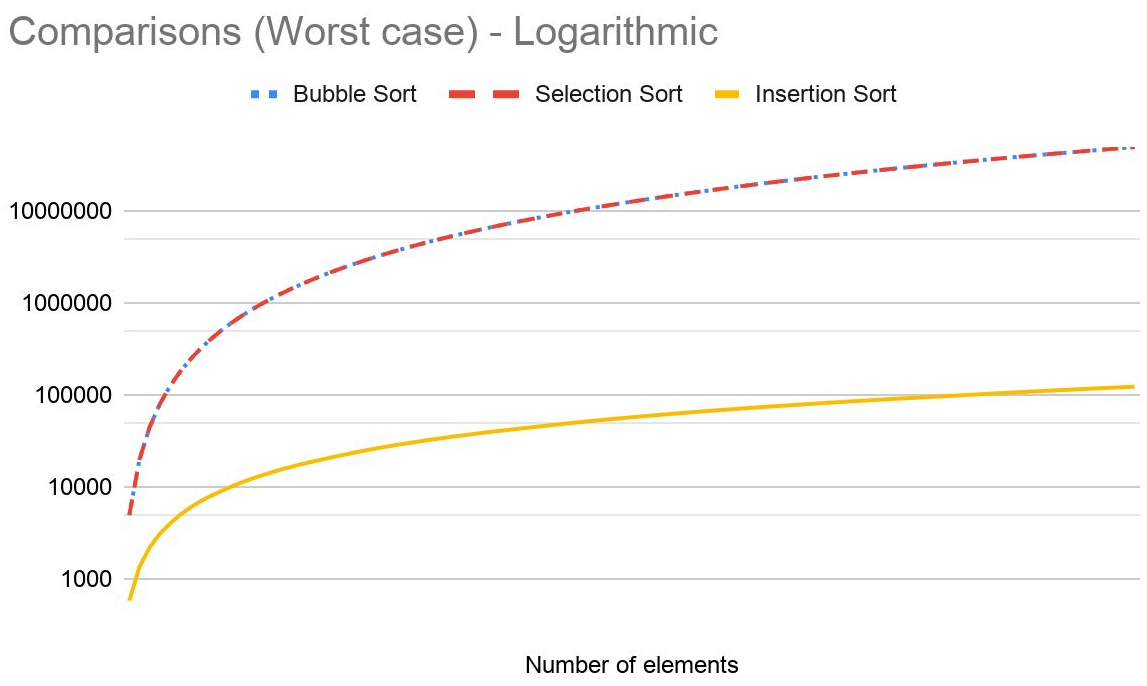

Total operations (Best case)
Bubble Sort
Selection Sort
Insertion Sort
50000000
40000000
30000000
20000000
10000000
0
Number of elements

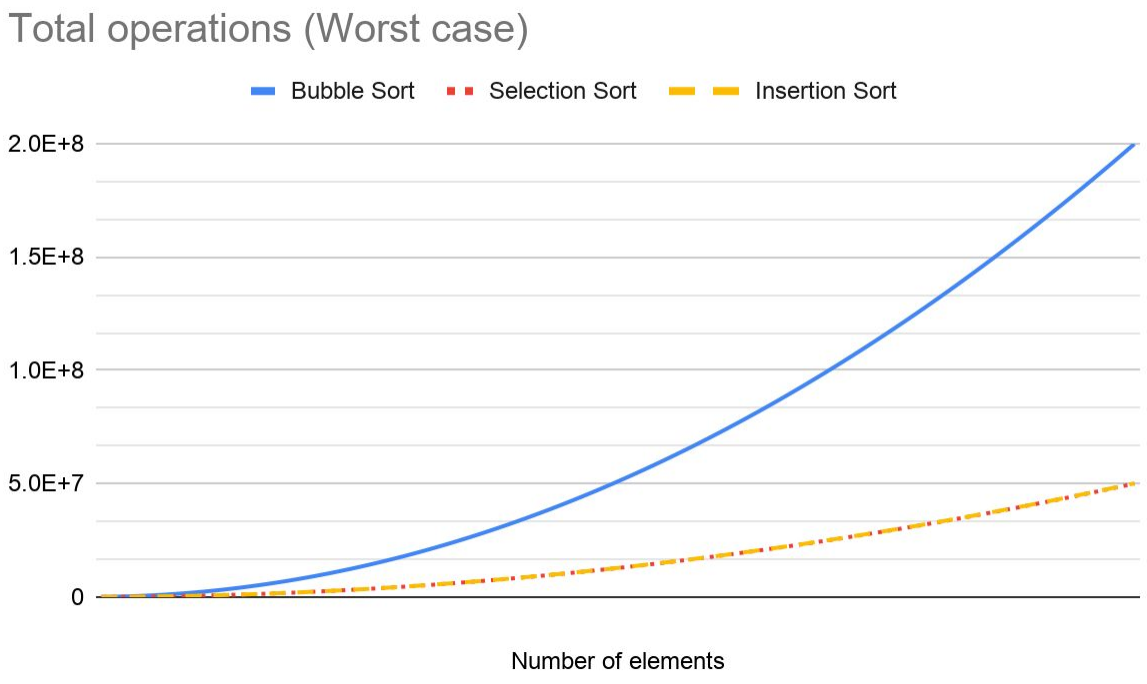

Total operations (Best case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
10000000
1000000
100000
10000
1000
100
Number of elements

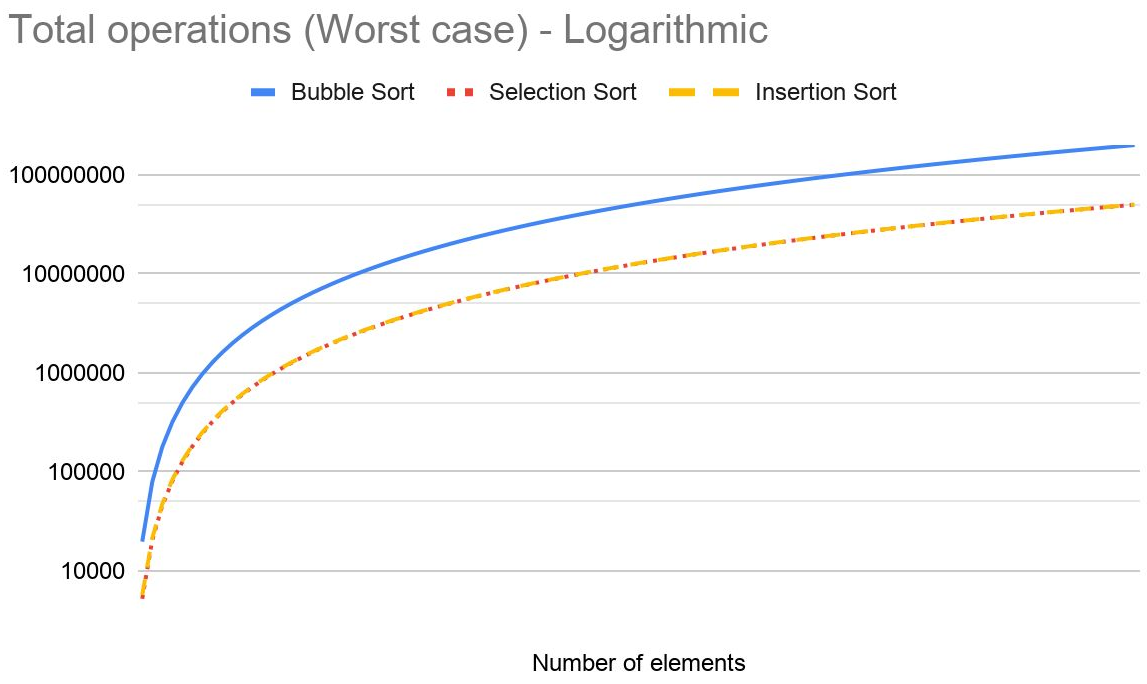

Assignments (Worst case)
Bubble Sort
Selection Sort
Insertion Sort
1.5E+8
1.0E+8
5.0E+7
0
Number of elements

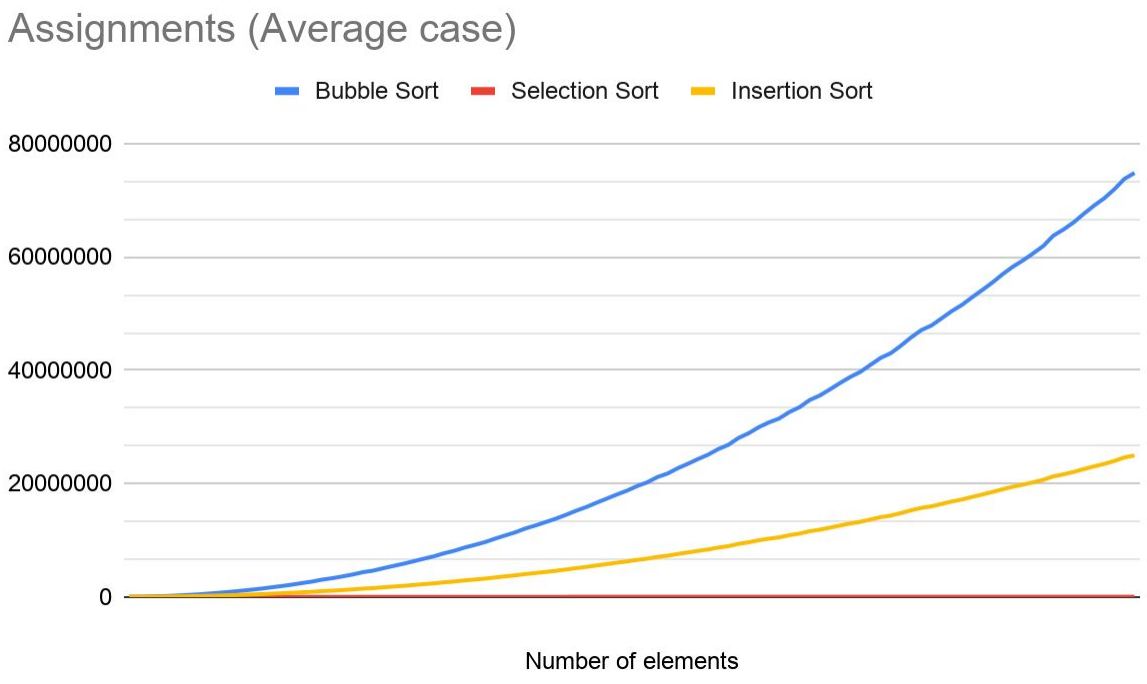

Assignments (Worst case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
100000000
10000000
1000000
100000
10000
1000
Number of elements

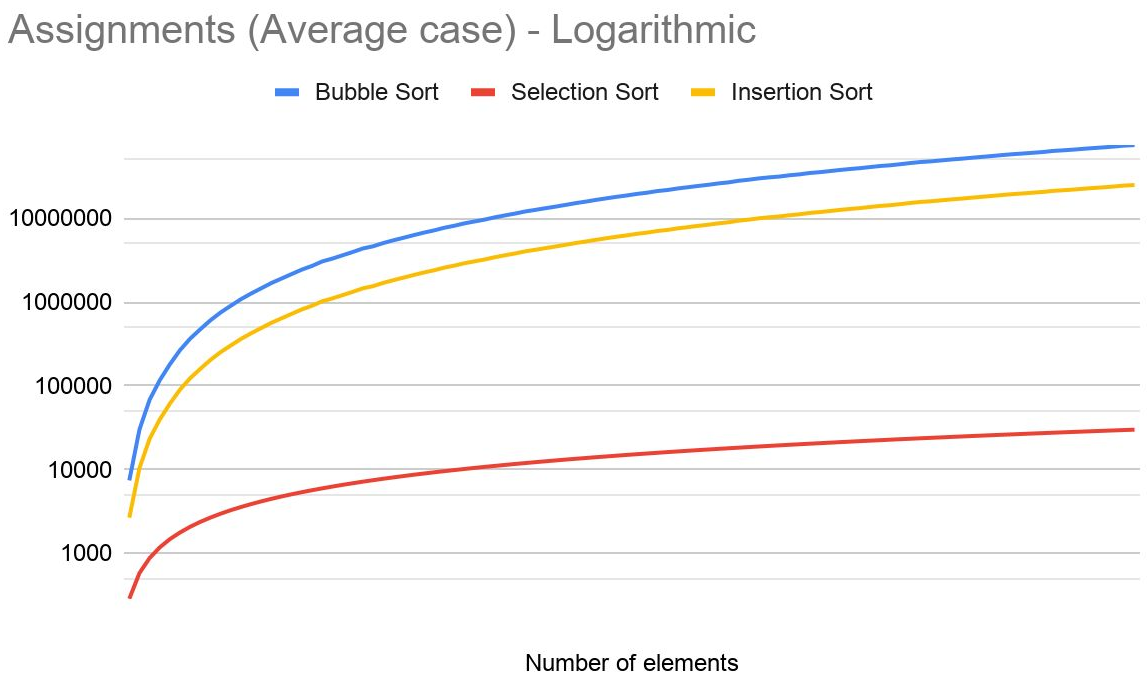

Comparisons (Worst case)
Bubble Sort
Selection Sort
Insertion Sort
50000000
40000000
30000000
20000000
10000000
0
Number of elements

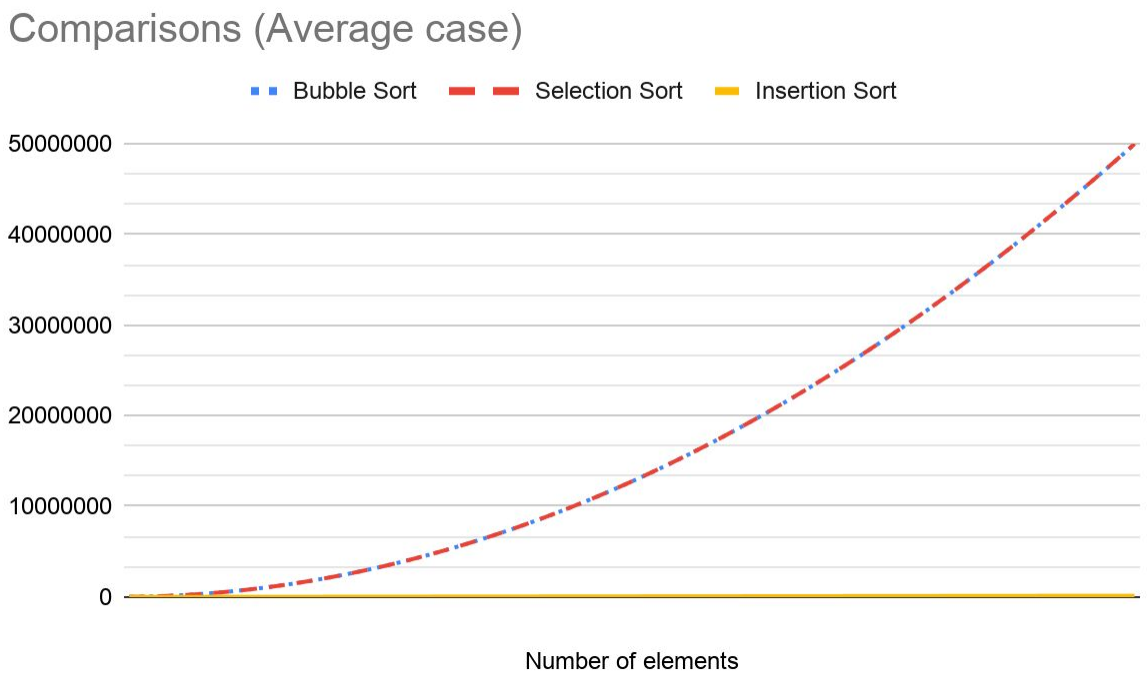

Comparisons (Worst case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
10000000
1000000
100000
10000
1000
Number of elements

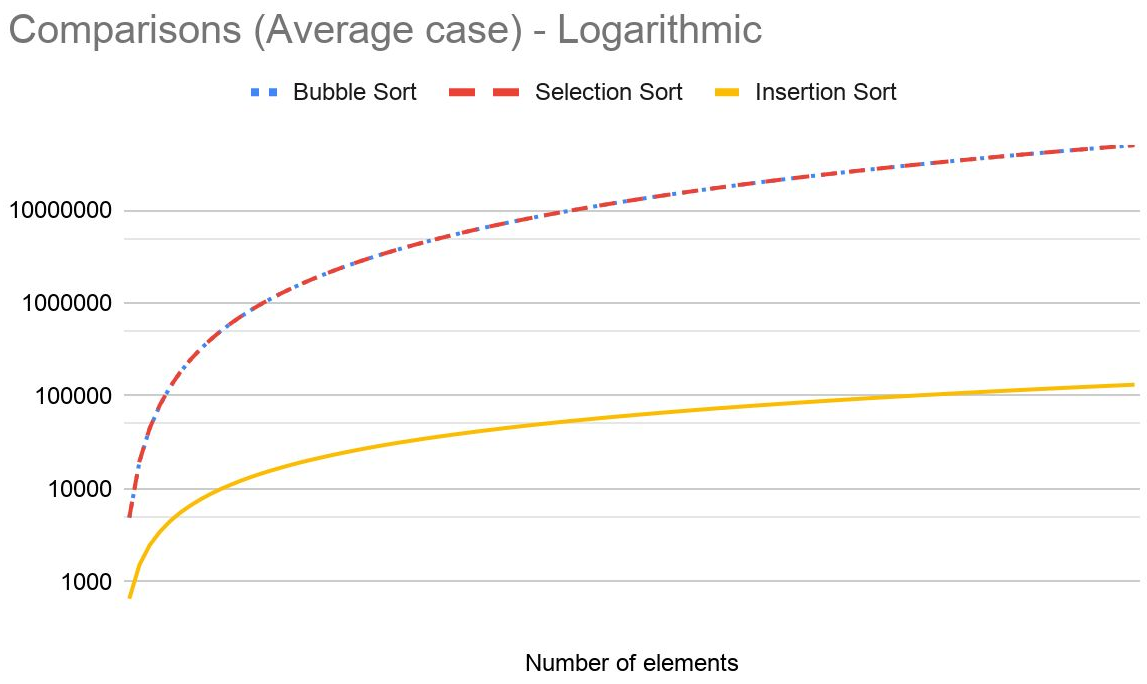

Total operations (Worst case)
Bubble Sort
Selection Sort
Insertion Sort
2.0E+8
1.5E+8
1.0E+8
5.0E+7
0
Number of elements

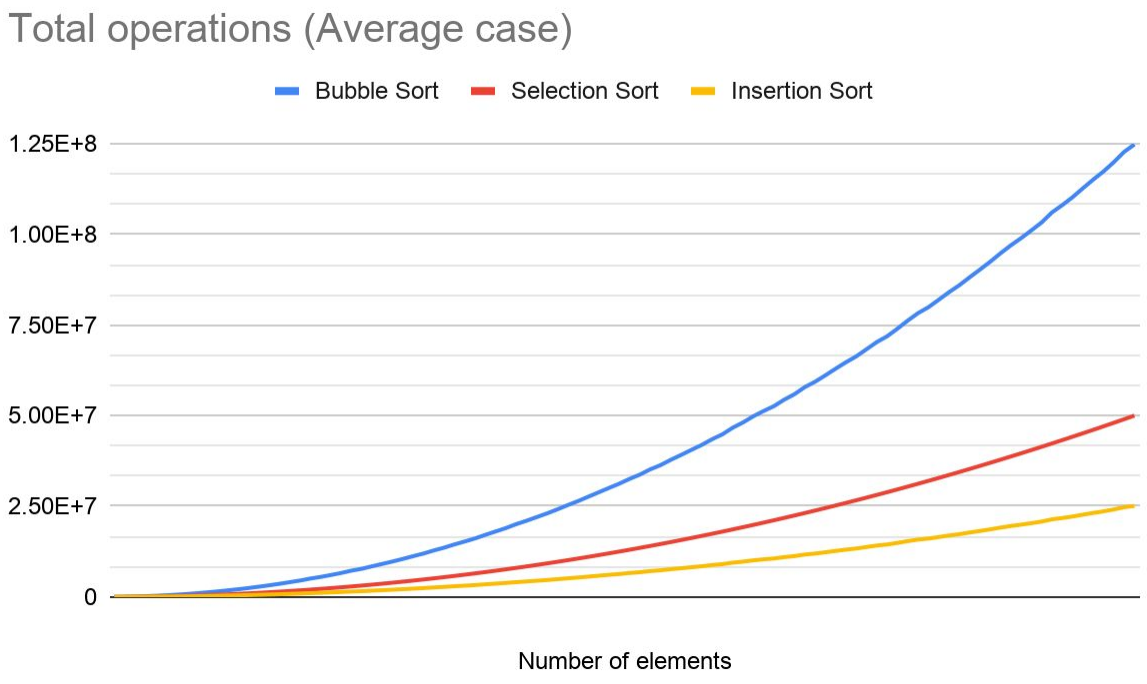

Total operations (Worst case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
100000000
10000000
1000000
100000
10000
Number of elements

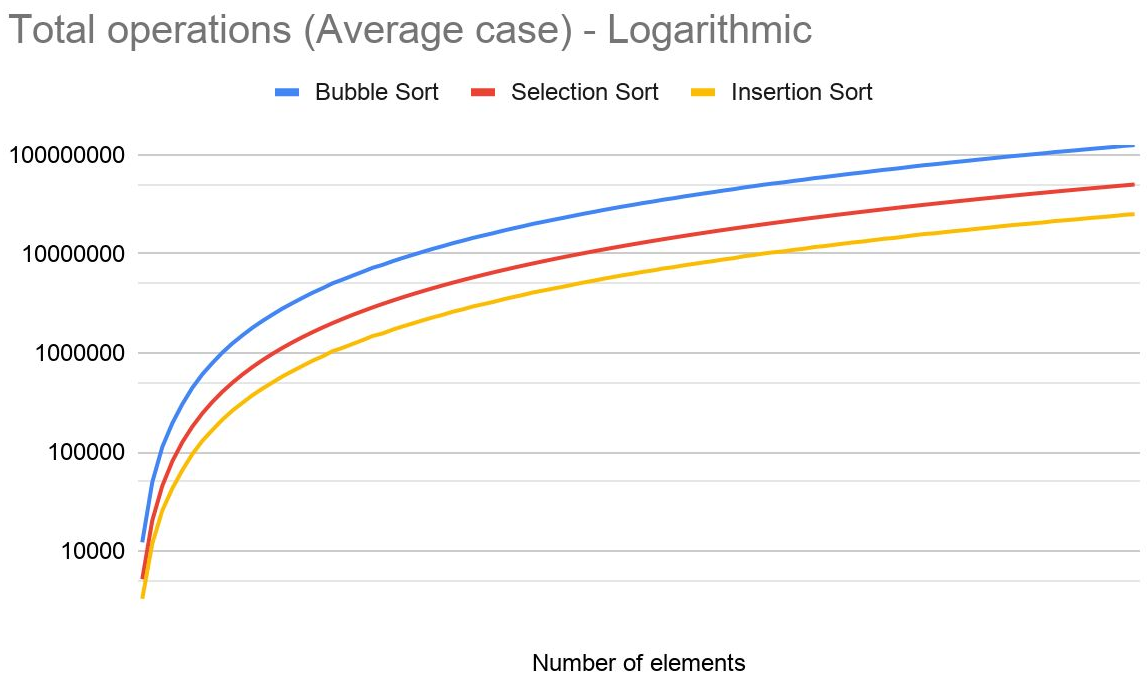

Assignments (Average case)
Bubble Sort
Selection Sort
Insertion Sort
80000000
60000000
40000000
20000000
0
Number of elements

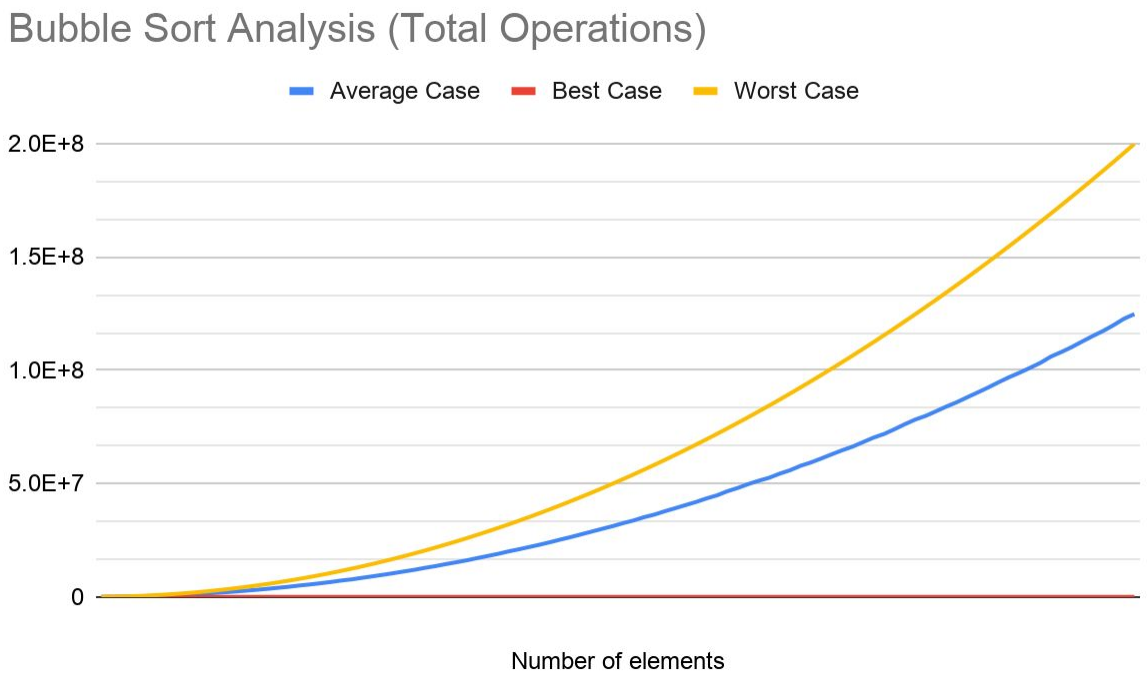

Assignments (Average case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
10000000
1000000
100000
10000
1000
Number of elements

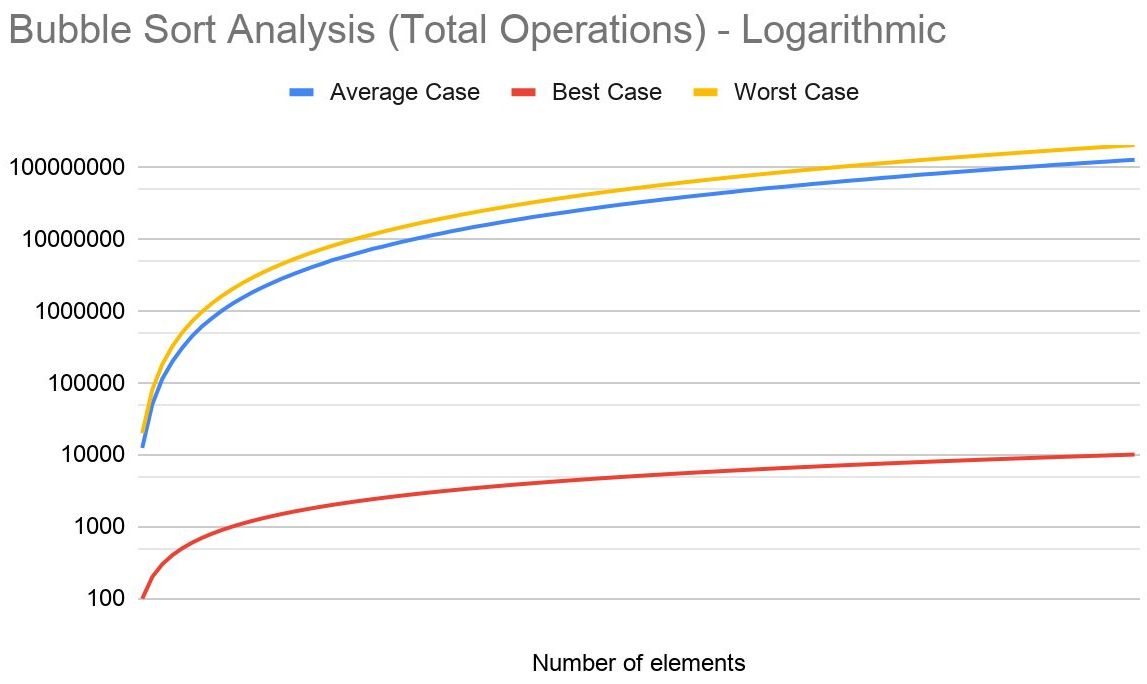

Comparisons (Average case)
Bubble Sort
Selection Sort
Insertion Sort
50000000
40000000
30000000
20000000
10000000
0
Number of elements

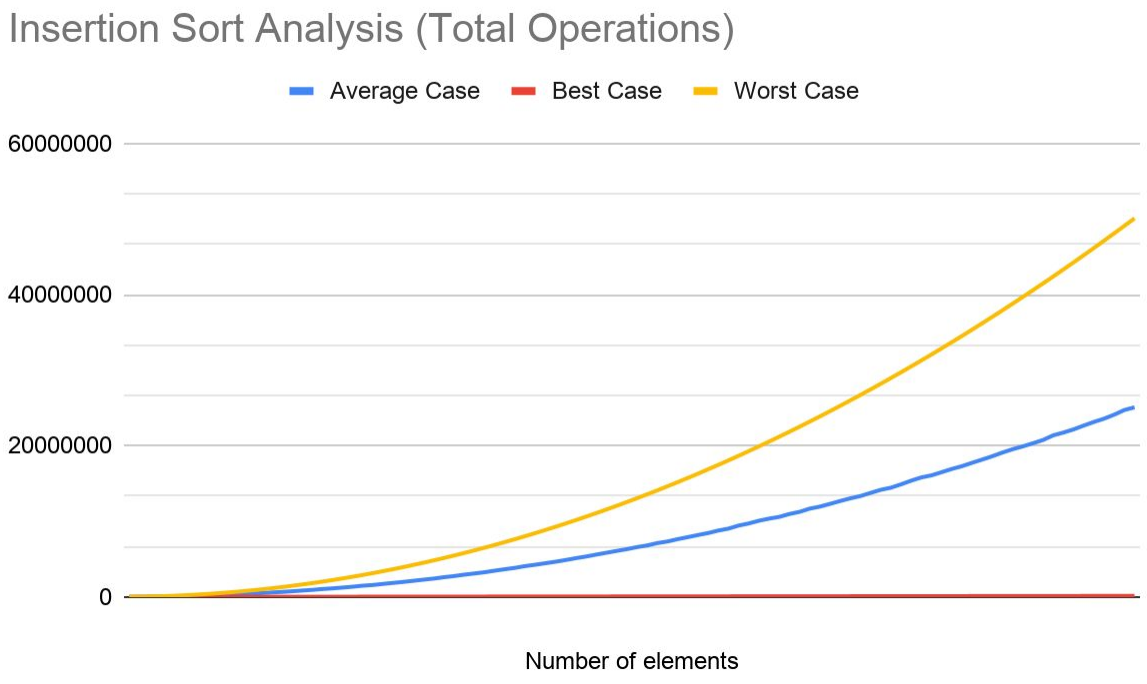

Comparisons (Average case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
10000000
1000000
100000
10000
1000
Number of elements

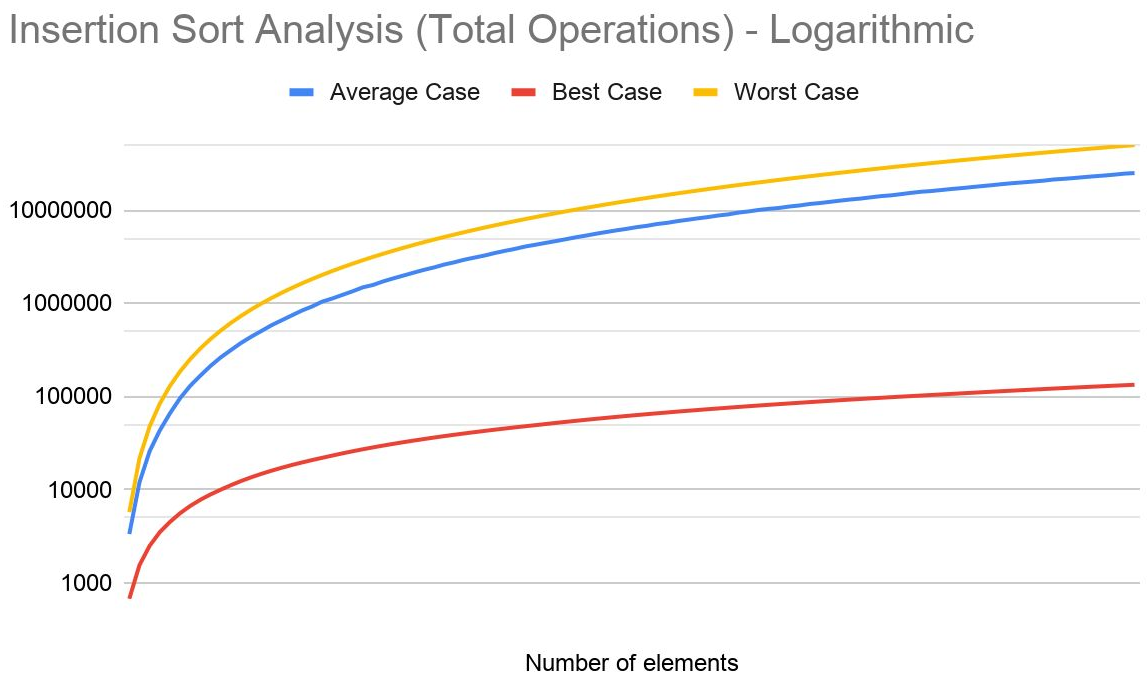

Total operations (Average case)
Bubble Sort
Selection Sort
Insertion Sort
1.25E+8
1.00E+8
7.50E+7
5.00E+7
2.50E+7
0
Number of elements

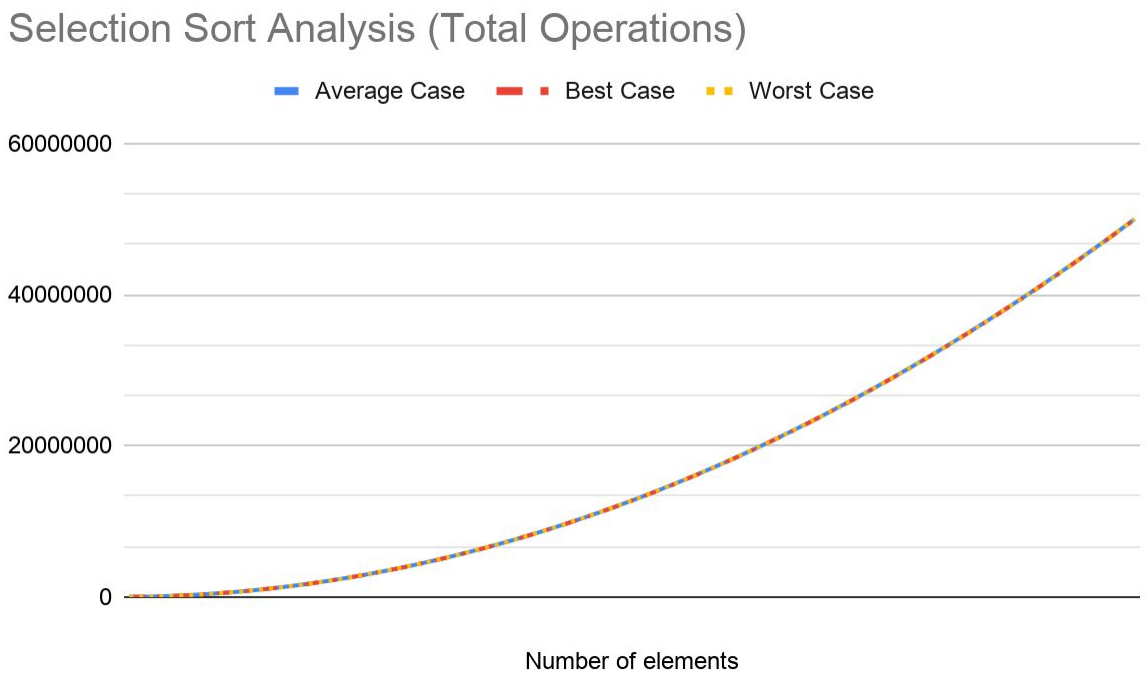

Total operations (Average case) - Logarithmic
Bubble Sort
Selection Sort
Insertion Sort
100000000
10000000
1000000
100000
10000
Number of elements

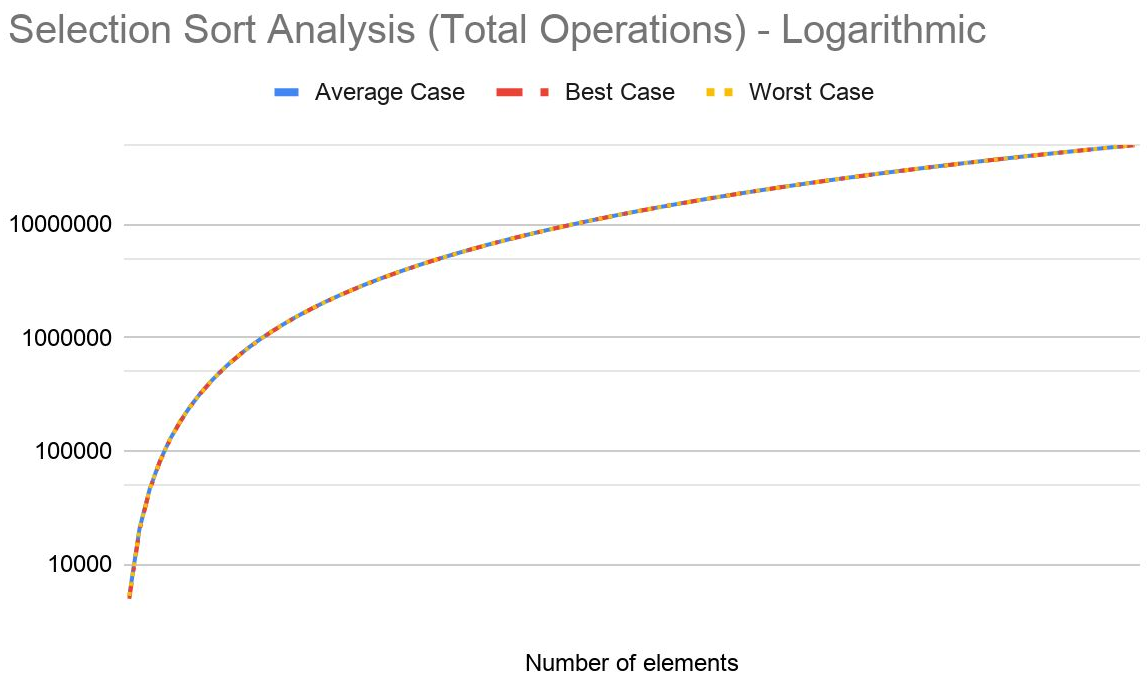

Bubble Sort Analysis (Total Operations)
Average Case
Best Case
Worst Case
2.0E+8
1.5E+8
1.0E+8
5.0E+7
0
Number of elements


Bubble Sort Analysis (Total Operations) - Logarithmic
Average Case
Best Case
Worst Case
100000000
10000000
1000000
100000
10000
1000
100
Number of elements


Insertion Sort Analysis (Total Operations)
Average Case
Best Case
Worst Case
60000000
40000000
20000000
0
Number of elements


Insertion Sort Analysis (Total Operations) - Logarithmic
Average Case
Best Case
Worst Case
10000000
1000000
100000
10000
1000
Number of elements


Selection Sort Analysis (Total Operations)
Average Case
Best Case
Worst Case
60000000
40000000
20000000
0
Number of elements


Selection Sort Analysis (Total Operations) - Logarithmic
Average Case
Best Case
Worst Case
10000000
1000000
100000
10000
Number of elements